



UNIVERSITY OF CAPE TOWN

COURSE CODE : CS2042S

COURSE NAME: MACHINE LEARNING

CSC2042S — Assignment 2 Report

Author:

Phumla . A. Khumalo

Student Number:

KHMPHU016

September 20, 2026

Contents

1	Model and Experimental Design	2
2	Hyperparameter Tuning	3
2.1	Search Space	3
2.2	Training and Model Selection	3
2.3	Tuning Results	3
3	Experimental Results	4
3.1	Baseline vs. Tuned Models	4
3.2	Weight Analysis	5
3.3	Class-Imbalance Experiments	5
3.4	Bilingual Training	5
3.5	Overall isiXhosa Results	6
4	Conclusion	7

1 Model and Experimental Design

The model is implemented as a single PyTorch `nn.Module` subclass, `MultinomialLogisticRegression` (`num_classes`), which wraps one linear layer (`nn.Linear`) and exposes exactly the three methods the assignment requires:

- `forward(x)` — returns raw logits for a batch (used internally by the training loop and by `nn.CrossEntropyLoss`, which expects unnormalised logits).
- `compute_probabilities(x)` — applies a row-wise softmax to the logits, giving a proper probability distribution over the classes for each input.
- `predict(x)` — returns the argmax class index per input, built on top of `compute_probabilities`.

Keeping the class this small (one linear layer, no hidden state beyond its weights) is a deliberate design choice: since the number of classes differs by language (6 for English, 5 for isiXhosa, 4 for chiShona) and the vocabulary size changes with every hyperparameter setting, the class is instantiated fresh for every experiment with the correct `input_dim/num_classes`, rather than trying to share one model object across configurations.

The rest of the pipeline is organised as small, composable functions rather than additional classes, since each one is a stateless transformation that is re-used identically across languages and experiments:

- `prepare_language(...)` — cleans text, fits a `CountVectorizer/TfidfVectorizer` and a `LabelEncoder` on the training split only, and returns a dictionary of ready-to-use train/dev/test tensors plus the fitted vectoriser and encoder (needed later for weight analysis and for re-vectorising held-out sets consistently).
- `train_model(...)` / `train_model_flexible(...)` — a mini-batch SGD training loop with early stopping on validation accuracy; the flexible variant additionally accepts an optional PyTorch `Sampler`, so the same loop drives both the ordinary experiments and the class-imbalance experiments (Section 3.3) without duplicating code.
- `evaluate_model(...)` — computes accuracy plus micro/macro precision, recall and F1, and renders a confusion matrix.
- `prepare_bilingual(...)` — the bilingual counterpart of `prepare_language`: fits one shared vectoriser/label-encoder on the concatenated training data of two languages, while keeping each language's own test set separately transformable.

This function-based design keeps the single model class fully general-purpose (it

has no knowledge of languages, sampling strategies, or vocabularies), while all of the experiment-specific logic — what data to feed it and how to sample it — lives outside the class and can be swapped freely.

2 Hyperparameter Tuning

2.1 Search Space

For every (language, feature-type) combination we ran a full grid search over four hyperparameters: learning rate (0.01, 0.1, 0.3), batch size (32, 64), vocabulary size / `max_features` (3000, 5000, 8000), and minimum document frequency `min_df` (0, 3, 5, 10) (0 implemented as the float 0.0, since scikit-learn requires an integer `min_df` to be ≥ 1). This is $3 \times 2 \times 3 \times 4 = 72$ configurations per combination, and $3 \text{ languages} \times 2 \text{ feature types} \times 72 = 432$ models trained in total.

2.2 Training and Model Selection

Each model was trained for up to 60 epochs with early stopping: validation accuracy is checked after every epoch, the best-so-far weights are checkpointed, and training stops once 5 consecutive epochs pass without improvement. This was chosen over a fixed epoch budget because the three languages differ hugely in training-set size (3309 / 1032 / 1288 documents for English / isiXhosa / chiShona) and a single fixed budget would either under-train the larger configurations or over-fit the smaller ones. For every (language, feature-type) group, the configuration with the highest validation accuracy was selected, and only that configuration’s performance was measured on the test set — the test set was never used to choose hyperparameters.

2.3 Tuning Results

Best configuration selected per (language, feature type):

Language	Features	lr	batch	vocab	min_df	Val Acc
English	BoW	0.3	32	8000	10	0.890
English	TF-IDF	0.3	32	8000	3	0.869
isiXhosa	BoW	0.3	32	3000	0	0.925
isiXhosa	TF-IDF	0.3	32	8000	5	0.857
chiShona	BoW	0.01	32	3000	10	0.908

Key observations from tuning:

1. **Feature type.** Bag-of-Words beat TF-IDF for all three languages on validation

accuracy. The gap was largest for isiXhosa (0.925 vs. 0.857) — with under 1100 training documents, TF-IDF’s real-valued, sub-1 feature magnitudes combined with plain SGD converged less reliably than BoW’s raw integer counts within the epoch/patience budget.

2. **Learning rate** was the strongest driver of convergence stability: $\text{lr} = 0.01$ with TF-IDF features frequently failed to leave a poor initial region in time, while $\text{lr} = 0.3$ converged reliably across almost every combination; BoW’s larger-magnitude features were more forgiving of a low learning rate.
3. **Vocabulary size and min_df** mattered more for the low-resource languages: isiXhosa’s and chiShona’s best configurations both used the smallest vocabulary tested (3000 features) with a non-trivial min_df, consistent with a small training set being unable to reliably estimate weights for a very large, sparse vocabulary. English’s best configuration used the full 8000-feature vocabulary, since its larger training set (3309 documents) could support it.
4. **Batch size** had comparatively little effect on final validation accuracy, mainly affecting the smoothness of the per-epoch loss curve.

3 Experimental Results

3.1 Baseline vs. Tuned Models

A default-hyperparameter baseline (`max_features= 5000`, `min_df= 3`, `lr= 0.1`, `batch= 64`) was trained for every language/feature combination before tuning, for comparison:

Language	Features	Val Acc	Test Acc	Test F1 (macro)
English	BoW	0.877	0.892	0.888
English	TF-IDF	0.852	0.847	0.839
isiXhosa	BoW	0.898	0.896	0.808
isiXhosa	TF-IDF	0.660	0.653	0.314
chiShona	BoW	0.892	0.921	0.923
chiShona	TF-IDF	0.876	0.900	0.898

Tuning gave the clearest benefit for isiXhosa + *TF-IDF* (macro-F1 0.314 $\rightarrow$ 0.529): the untuned default learning rate was simply too small for TF-IDF features on this dataset, so tuning mostly rescued a configuration that was failing to train at all, rather than incrementally polishing an already-working one.

3.2 Weight Analysis

Inspecting the learned softmax weights for the best English model, the top positive words per class are overwhelmingly sensible:

business → business, prices, bank, workers, price;

entertainment → show, music, film, festival;

health → health, covid, patients, hospital, nhs;

politics → sunak, scottish, government, welsh, wales;

sports → sport, team, olympic, world;

technology → data, tiktok, cyber, app, tech.

Two points are worth flagging: the *politics* class leans heavily on specific UK entities (Sunak, Scotland, Wales) rather than a general notion of “politics” — a symptom of this English split being sourced largely from UK/BBC news, which may not generalise to political news from other regions. The *technology* class also gives non-trivial positive weight to the common modal verb “can”, which is almost certainly a frequency artefact of this training sample rather than a meaningful topic indicator.

3.3 Class-Imbalance Experiments

isiXhosa’s training set is heavily imbalanced (entertainment: 350, sports: 347, politics: 215, health: 70, business: 50). Starting from the tuned isiXhosa baseline, we tested inverse-frequency upsampling (`WeightedRandomSampler`, sweeping the reweighting strength α) and majority-class downsampling (capping every class to a multiple of the smallest class, sweeping the ratio):

Strategy	Best setting	Val Acc	Test Acc	Test F1 (macro)
No sampling (baseline)	—	0.925	0.882	0.770
Upsampling	alpha = 1.5	0.925	0.892	0.814
Downsampling	ratio = 2.0 (420/1032 kept)	0.898	0.865	0.764

3.4 Bilingual Training

Upsampling gave a real improvement (+4.4 macro-F1 points) by letting the model see minority-class examples far more often without discarding any data. Downsampling, which discarded roughly 60% of the training set to force balance, was neutral-to-slightly-harmful — for a dataset this small, losing that much training signal outweighed the benefit of a balanced label distribution.

We trained one shared model on isiXhosa + *chiShona* (related, similarly-sized) and isiXhosa + *English* (unrelated, English $> 3\times$ larger), sweeping the shared vocabulary size and evaluating on isiXhosa’s own test set:

Pairing	Best vocab	Val Acc	isiXhosa Test Acc	isiXhosa Test F1 (macro)
isiXhosa only (baseline)	—	0.925	0.882	0.770
isiXhosa + chiShona	12000	0.904	0.896	0.813
isiXhosa + English	8000	0.876	0.892	0.664

We had predicted isiXhosa + *chiShona* would help more, since the two languages are linguistically related and similarly sized, while English ($> 3\times$ isiXhosa’s training size) would dominate a shared vocabulary. The results confirm this: validation accuracy for isiXhosa + *chiShona* improved monotonically as the shared vocabulary grew to 12000 features (the two languages claim comparable shares of the vocabulary — ~ 6400 tokens each), whereas isiXhosa + *English* peaked at 8000 features and then worsened at 12000, as English’s share of the added vocabulary slots grew far faster than isiXhosa’s (9413 vs. 4605 tokens at 12000 features) — a direct illustration of a larger language crowding out a smaller one’s vocabulary. Test accuracy alone makes the two pairings look similar (0.896 vs. 0.892), but macro-F1 exposes a large real difference (0.813 vs. 0.664): pairing isiXhosa with an unrelated, much larger language badly hurts its minority classes even though overall accuracy barely moves.

3.5 Overall isiXhosa Results

Approach	Test Acc	Test F1 (macro)
Upsampling (alpha= 1.5)	0.892	0.814
Bilingual isiXhosa + chiShona	0.896	0.813
Monolingual tuned baseline	0.882	0.770
Downsampling (ratio= 2.0)	0.865	0.764
Bilingual isiXhosa + English	0.892	0.664

Upsampling and bilingual isiXhosa + *chiShona* training are effectively tied for the best result (macro-F1 0.814 vs. 0.813), both roughly 4.4 points above the plain tuned baseline. Both winning strategies share a common mechanism: they give the model more effective exposure to isiXhosa’s smallest classes (business: 50, health: 70 examples) without discarding any of isiXhosa’s own training data. Both underperforming strategies (downsampling, isiXhosa + *English*) instead reduce the effective information available about those same minority classes, either by deleting examples outright or by diluting isiXhosa’s vocabulary in a much larger shared feature space.

4 Conclusion

A simple, well-tuned multinomial logistic regression model already reaches ~ 0.88 – 0.92 test accuracy on all three MasakhNEWS languages using nothing more than Bag-of-Words features. Bag-of-Words consistently beat TF-IDF, and vocabulary size/min_df mattered most for the two lower-resource languages. For isiXhosa specifically, class imbalance is the main limiting factor on macro-F1, and it can be meaningfully addressed either by upsampling minority classes or by jointly training with a linguistically related, similarly-sized language (chiShona) — but not by downsampling, and not by joint training with a much larger, unrelated language (English), both of which reduce the information available about isiXhosa’s smallest categories.

AI Usage Statement

Generative AI tools were used during the development of this assignment for coding support and writing-related assistance. In particular, ChatGPT was used to help understand aspects of the assignment requirements and PyTorch code, explain programming concepts, assist with debugging errors encountered during implementation, and provide feedback on the organisation and clarity of the notebook and report.

The AI-generated feedback was used as a support tool rather than as a replacement for my own implementation and understanding. I reviewed, tested, and adapted the suggestions to fit the requirements of the assignment and verified the resulting code through my own experiments. The final implementation, experimental results, analysis, and conclusions were therefore reviewed and determined by me.